Vistas, DBML y DDL en SQLite

Consultas guardadas, diagramas como código, el DDL en SQLite y los archivos `.sql`

Semana 08 - Sesión 16

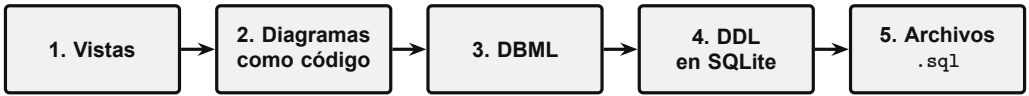
Rel Guzman



Logro de la sesión

Al finalizar, podrás **crear y borrar una vista**, **escribir el diseño** de una base de datos en **DBML** para ver su diagrama, reconocer **qué cambia en SQLite** al crear, modificar y borrar tablas, y explicar **qué es un archivo** `.sql`.

Lo de hoy: cinco paradas



- Ya sabes **crear tablas** y **consultarlas**. Hoy no hay instrucciones difíciles.
- Hoy vemos **ayudas**: guardar una consulta, dibujar el diseño escribiendo y guardar todo en un archivo.

Ojo con esto

Todo es **introductorio**. La meta es entender **la idea** de cada cosa.



Vistas

La misma consulta, una y otra vez

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

El encargado de bebidas pide esto **todos los días**:

```
SELECT nombre, precio
FROM producto
WHERE categoria = 'bebidas';
```

Escribirla cada vez **cansa**, y es fácil **equivocarse**.

La idea

Le ponemos **un nombre** a esa consulta y el gestor **la guarda**. Eso es una **vista**.

En palabras simples

Es como **guardar un contacto**: en vez de marcar el número completo cada vez, buscas «Mamá».

`CREATE VIEW`: una consulta con nombre

```
CREATE VIEW bebidas AS
SELECT nombre, precio FROM producto
WHERE categoria = 'bebidas';
```

- el **nombre** que le pones
- la **consulta** que se guarda
- con su condición y todo

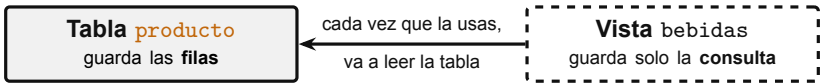
Desde ahora se usa **como si fuera una tabla**:

```
SELECT * FROM bebidas;
```

nombre	precio
Gaseosa	3.00
Agua	1.50
Leche	3.80

El `CREATE VIEW` se escribe **una sola vez**. Es igual en MySQL, SQL Server y SQLite.

Una vista no guarda datos: es una ventana



Llega un producto nuevo a la **tabla**:

```
INSERT INTO producto
VALUES (7, 'Chicha', 'bebidas', 2.50, 15);
```

nombre	precio
Gaseosa	3.00
Agua	1.50
Leche	3.80
Chicha	2.50

La Chicha aparece **sola** en la vista. Nadie tocó `bebidas`.

En palabras simples

Una **ventana** no guarda el paisaje: te muestra lo que hay afuera **en este momento**.

`DROP VIEW`: borrar la vista, no los datos

```sql
DROP VIEW bebidas;
```

Se borra **la consulta guardada**.

La tabla `producto` sigue con **todas sus filas**.

```sql
DROP TABLE producto;
```

Se borra **la tabla con sus datos**.

Esto sí es grave.

En palabras simples

Borrar un **contacto** del celular no desaparece a la persona. Borrar una vista no desaparece los datos.

¿Y si quiero **cambiar** una vista? Lo más simple: la **borras** y la **creas de nuevo**.

¿Cuándo es útil una vista?

Para no repetir

Una consulta que se usa seguido se escribe **una vez**.

Para simplificar

Detrás puede haber algo largo; quien la usa ve algo **sencillo**.

Para ocultar

Muestra solo **algunas columnas**: el precio sí, el costo no.

¿Y cuándo no?

Si la consulta la vas a usar **una sola vez**, no hace falta una vista.

Cómo se ve en la realidad

En un colegio, la secretaría usa una vista con el **nombre y el teléfono** de cada alumno. Las notas y los pagos no aparecen ahí.

Práctica 1 Vistas sobre la bodega

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Se crea esta vista:

```
CREATE VIEW baratos AS
SELECT nombre, precio
FROM producto
WHERE precio < 3;
```

Pregunta	Respuesta
¿Qué nombres muestra `SELECT * FROM baratos;`?	
La Gaseosa baja a 2.50. ¿Aparece ahora en `baratos`?	
Se ejecuta `DROP VIEW baratos;`. ¿Cuántas filas tiene `producto`?	

Para escribir: crea la vista `agotados`, con los productos cuyo stock es 0.

Diagramas como código

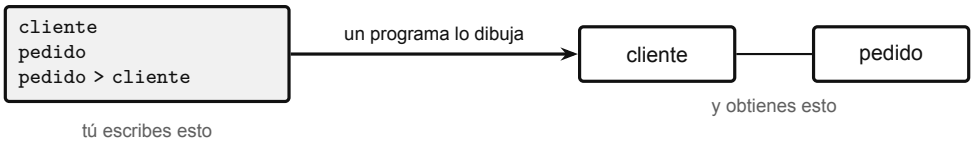
Dos maneras de hacer un diagrama

Con el mouse

- Arrastras cajas y las unes con líneas, **a mano**.
- Si agregas una tabla, toca **reacomodar** todo.

Con código

- **Escribes** qué tablas hay y cómo se conectan.
- Un programa **dibuja el diagrama** por ti.



En palabras simples

Es la diferencia entre la **foto de un plato** y su **receta**. En la receta, cambiar un ingrediente es cambiar **una línea**.

¿Por qué escribir los diagramas?

- **Es rápido de cambiar:** corriges una línea y el dibujo se rehace solo.
- **Es texto:** se copia, se pega en un chat y se guarda junto al proyecto.
- **Lo puede escribir una IA:** le cuentas el caso, te da el texto y tú lo revisas.

Lenguaje	Sirve para
Mermaid	diagramas de flujo
PlantUML	diagramas UML
DBML	**bases de datos**

hay varios lenguajes para dibujar con texto



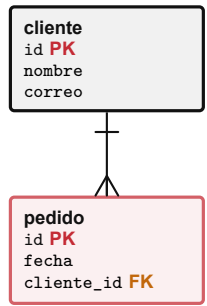
DBML

DBML: describir una base de datos con texto

```
Table cliente {
   id int [pk]
   nombre varchar(80)
   correo varchar(120)
}

Table pedido {
   id int [pk]
   fecha date
   cliente_id int
}

Ref: pedido.cliente_id > cliente.id
```



el diagrama que sale de ese texto

DBML es un lenguaje pequeño y gratuito, hecho solo para **describir tablas y sus relaciones**. Se lee casi sin haberlo estudiado.

¿Por qué DBML, si ya sabemos SQL?

SQL

```
CREATE TABLE pedido (
   id INT PRIMARY KEY,
   fecha DATE NOT NULL,
   cliente_id INT,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);
```

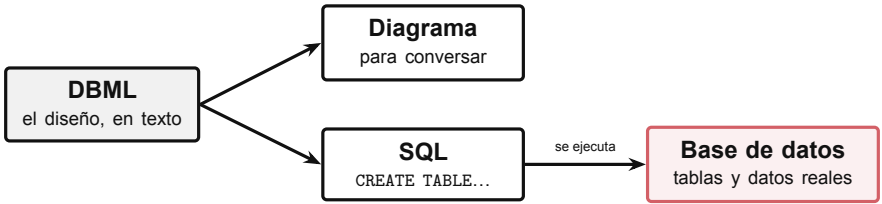
DBML

```
Table pedido {
   id int [pk]
   fecha date [not null]
   cliente_id int
}

Ref: pedido.cliente_id > cliente.id
```

- **Es más corto:** menos palabras y sin comas que se olvidan.
- **Es uno solo:** el SQL cambia de un gestor a otro; el DBML no.
- **Se vuelve diagrama al instante:** sirve para conversar el diseño **antes** de crear nada.

DBML no reemplaza a SQL: lo ayuda



DBML sí

- Describe tablas y relaciones.
- Genera el diagrama.
- Genera el `CREATE TABLE`.

DBML no

- **No crea** la base de datos.
- **No guarda** datos.
- **No consulta**: no hay `SELECT`.

En palabras simples

DBML es el **plano** de la casa; SQL es **la obra**. El plano ayuda a pensar, pero nadie vive en un plano.

Una tabla en DBML, pieza por pieza

```
Table cliente {
  id int [pk]
  nombre varchar(80) [not null]
  correo varchar(120) [unique]
}
```

- `Table`, el **nombre** y una llave que abre
- una columna por línea: **nombre** y **tipo**
- entre **corchetes**, las reglas
- **sin coma** al final de la línea
- la llave que cierra

El orden es el mismo de `CREATE TABLE`: **nombre de la columna, tipo y reglas**.

Las reglas entre corchetes

En DBML	En SQL	Qué significa
`pk`	`PRIMARY KEY`	identifica a cada fila
`increment`	`AUTO_INCREMENT`	el gestor numera solo
`not null`	`NOT NULL`	no se puede dejar vacío
`unique`	`UNIQUE`	no se puede repetir

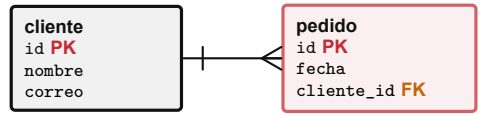
```
id int [pk, increment]
```

Son las **mismas reglas** que ya conoces, más cortas. Si hay varias, se separan con **comas**.

Las relaciones: `Ref`

```
Ref: pedido.cliente_id > cliente.id
```

- Se lee: «`cliente_id` de `pedido` **apunta** al `id` de `cliente`».
- Es la **clave foránea**, en una sola línea.



un cliente, muchos pedidos

Símbolo	Relación	Ejemplo
`>`	muchos a uno	`pedido.cliente_id > cliente.id`
`-`	uno a uno	`usuario.id - perfil.usuario_id`

Truco: el > se abre hacia el lado «**muchos**», igual que la pata de gallo.

Práctica 2 Lee este DBML

```
Table categoria {
   id int [pk, increment]
   nombre varchar(80) [unique]
}

Table producto {
   id int [pk, increment]
   nombre varchar(80) [not null]
   precio decimal(10,2)
   categoria_id int
}

Ref: producto.categoria_id > categoria.id
```

Pregunta	Resp.
¿Cuántas tablas hay?	_____
¿Qué columna es clave foránea?	_____
¿Pueden repetirse los nombres de categoría?	_____
¿Cuál tabla está del lado «muchos»?	_____

Dibuja en tu cuaderno el diagrama: dos cajas y la línea que las une.

Herramientas

Herramientas que usan DBML

Herramienta	Para qué sirve
dbdiagram.io	Escribes DBML y **ves el diagrama** al lado. Es la que usaremos.
dbdocs.io	Convierte el DBML en una **página web** que explica la base de datos.
Extensiones de VS Code	Muestran el diagrama **dentro del editor**.

DBML es **abierto**: cualquier herramienta puede leerlo. Tu diseño no queda atrapado en un solo programa.

dbdiagram.io en tres pasos



- Se usa desde el navegador: **no se instala nada**.
- Al exportar a SQL eliges el gestor, por ejemplo **MySQL**.

Ojo con esto

En la lista **no aparece SQLite**: el SQL que sale hay que **ajustarlo un poco**. Eso vemos en la siguiente parte.

Del enunciado al diagrama

Un colegio registra a sus **alumnos** y las **matrículas** de cada uno.

alumno

- `id`: clave primaria
- `nombre`: obligatorio
- `correo`: no se repite

matricula

- `id`: clave primaria
- `fecha`: obligatoria
- `alumno_id`: apunta al alumno

1. Escribe en DBML las **dos tablas** y la **relación**.
2. Pégalo en `dbdiagram.io` y mira el diagrama.
3. Exporta a **MySQL** y guarda ese SQL: lo usarás en la práctica 4.

Universidad Tecnológica del Perú

DDL en SQLite

¿Por qué SQLite?



- Toda la base de datos es **un solo archivo**: se copia y se envía como una foto.
- **No se instala** nada ni se crean usuarios.
- Está **en todas partes**: en tu celular, en tu navegador y dentro de Python.

Ojo con esto

Por ser tan pequeño, SQLite **simplifica**. Las instrucciones ya las conoces: solo veremos **qué cambia**.

`CREATE TABLE` en SQLite

MySQL

```
CREATE TABLE cliente (
   id INT PRIMARY KEY AUTO_INCREMENT,
   nombre VARCHAR(80) NOT NULL,
   registro DATE
);
```

SQLite

```
CREATE TABLE cliente (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   nombre TEXT NOT NULL,
   registro TEXT
);
```

- **Pocos tipos:** `INTEGER` para números enteros, `REAL` para decimales y `TEXT` para todo lo demás, incluso las fechas.
- **No hay** `CREATE DATABASE`: la base de datos **es el archivo**.
- Lo demás (`PRIMARY KEY`, `NOT NULL`, `UNIQUE`) se escribe **igual**.

¿Por qué? SQLite quiere ser pequeño y simple: con tres tipos le alcanza.

`FOREIGN KEY` en SQLite: hay que encenderla

```
PRAGMA foreign_keys = ON;

CREATE TABLE pedido (
   id INTEGER PRIMARY KEY,
   fecha TEXT,
   cliente_id INTEGER,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);
```

- Se escribe **igual** que en MySQL.
- Pero viene **apagada**: hay que encenderla con el `PRAGMA`.

Un pedido del cliente 99, que no existe	Qué pasa
Sin el `PRAGMA`	Se guarda, y apunta a nadie
Con el `PRAGMA`	SQLite lo **rechaza**

En palabras simples

Es como una **alarma** que viene instalada pero apagada: si no la enciendes, no avisa.

`ALTER TABLE` en SQLite: solo lo básico

Sí se puede

```sql
-- agregar una columna
ALTER TABLE cliente
    ADD COLUMN telefono TEXT;

-- cambiarle el nombre a una columna
ALTER TABLE cliente
    RENAME COLUMN correo TO email;
```

No se puede

- **Cambiar el tipo** de una columna.
- **Agregar una clave foránea** a una tabla que ya existe.

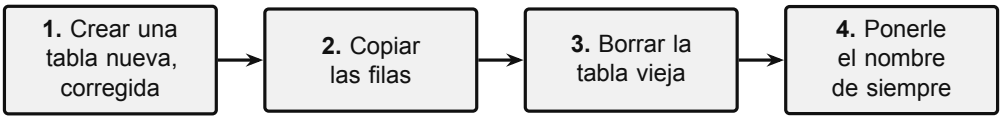
Ojo con esto

En SQLite, las claves foráneas se escriben **al crear la tabla**. Después ya no se pueden agregar.

¿Por qué? Para seguir siendo pequeño, SQLite solo trae los cambios más sencillos.

¿Y si necesito justo lo que no se puede?

Se **rehace la tabla**, como quien pasa un cuaderno en limpio:



Lo más fácil mientras aprendes

Corrige el `CREATE TABLE` en tu archivo `.sql` y **crea la base de nuevo**. Para eso sirve el archivo que veremos al final.

`DROP TABLE` en SQLite

```sql
DROP TABLE pedido;
```

Se escribe **igual** que en MySQL: borra la tabla **con todos sus datos**.

Quiero…	MySQL	SQLite
Borrar una tabla	`DROP TABLE pedido;`	`DROP TABLE pedido;`
Vaciar una tabla	`TRUNCATE TABLE pedido;`	`DELETE FROM pedido;`
Borrar toda la base	`DROP DATABASE tienda;`	**borrar el archivo**

Ojo con esto

Borra primero la tabla que **apunta** (`pedido`) y después la **apuntada** (`cliente`).

De MySQL a SQLite

Este SQL salió de dbdiagram para MySQL. Escribe cómo queda **cada línea** en SQLite.

En MySQL	En SQLite
`CREATE DATABASE colegio;`	
`id INT PRIMARY KEY AUTO_INCREMENT,`	
`nombre VARCHAR(80) NOT NULL,`	
`fecha DATE NOT NULL,`	
`TRUNCATE TABLE matricula;`	

Pista: una de las líneas **no existe** en SQLite. ¿Y qué línea falta al comienzo para encender las claves foráneas?

Archivos .sql

Un archivo `.sql`: instrucciones guardadas

tienda.sql

```sql
-- Tienda: tablas y datos de prueba
CREATE TABLE cliente (
   id INTEGER PRIMARY KEY,
   nombre TEXT NOT NULL
);
INSERT INTO cliente (nombre) VALUES ('Ana');
INSERT INTO cliente (nombre) VALUES ('Luis');
```

- Es un archivo de **texto** con instrucciones SQL.
- El gestor las ejecuta **de arriba hacia abajo**.
- Se abre con **cualquier editor**.

Es lo mismo que has escrito hasta hoy, pero **guardado** para no perderlo.

¿Para qué sirve un archivo `.sql`?

Repetir

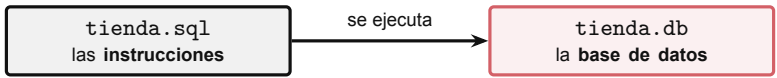
Crea la base completa **desde cero**, las veces que haga falta.

Compartir

Lo envías y la otra persona obtiene **la misma** base.

Respaldar

Queda una **copia** de las tablas y de los datos.



En palabras simples

El `.sql` es la **receta**; la base de datos es **el plato**. Con la receta lo vuelves a preparar cuando quieras.

Preguntas



Cierre: lo de hoy en una tabla

	En una frase
Vista	Una consulta guardada con nombre. No guarda datos.
Diagrama como código	Un texto que un programa convierte en dibujo.
DBML	El plano de la base de datos: ayuda a SQL, no lo reemplaza.
DDL en SQLite	Lo mismo de siempre, más simple: pocos tipos y un archivo.
Archivo `.sql`	La receta para crear la base de datos otra vez.

El camino completo: **piensas** en DBML, lo **ves** como diagrama, lo **exportas** a SQL, lo **guardas** en un `.sql` y lo **ejecutas**.

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.

Holistics. (s. f.). *DBML: Database Markup Language*. `https://dbml.dbdiagram.io`

SQLite. (s. f.). *SQL as understood by SQLite*. `https://www.sqlite.org/lang.html`



UTP Universidad Tecnológica del Perú